

MEJORES PRÁCTICAS · GUÍA RÁPIDA

8 prácticas reales para usar Claude sin perder el hilo.

Documentadas por Anthropic. Con comandos, ejemplos y qué escribirle exactamente a Claude en cada caso.

-
- 1 `/clear` — cuándo y por qué reiniciar la sesión
 - 2 Modo plan con Shift+Tab antes de tocar código
 - 3 Cómo darle un comando de verificación propio
 - 4 Anatomía de un prompt que sí funciona
 - 5 `CLAUDE.md` y el comando `/init` explicados
 - 6 Qué preguntarle cuando alucina (no lo corrijas)
 - 7 Cómo dividir tareas largas en fases verificables
 - 8 Sesiones paralelas: el truco del segundo par de ojos

PRÁCTICAS 1 – 2

Controla el contexto antes de que te controle a ti

El modelo no olvida lo que le dijiste hace 10 mensajes. Eso puede salvarte o hundirte dependiendo de cómo lo manejes.

1 NO REPITAS LA CORRECCIÓN MÁS DE DOS VECES

Si Claude falla dos veces seguidas en lo mismo, el contexto ya está contaminado. Cada mensaje nuevo se interpreta a través de los errores anteriores — corregirlo una tercera vez casi siempre empeora las cosas.

Qué hacer exactamente:

```
# 1. En Claude Code: escribe este comando en el chat
/clear
# Esto borra TODO el contexto de la sesión actual

# 2. En Claude.ai (web): abre una conversación nueva
# No uses 'continuar conversación' – empieza thread limpio

# 3. Después del clear, da contexto mínimo de nuevo:
Estoy trabajando en [archivo]. El problema es [X].
Necesito que [acción específica].
```

Por qué funciona: Anthropic documenta que los modelos tienen sesgo hacia sus propias respuestas recientes. Si Claude dijo que algo era de una forma y estaba mal, tenderá a defender esa versión aunque le corrijas. /clear elimina ese sesgo por completo.

TIEMPO: 30 SEG

2 EXPLORA → PLANEA → CODEA (EN ESE ORDEN)

El error más común: pedirle cambios sin que Claude entienda el contexto. El resultado es que resuelve el problema equivocado con código perfectamente escrito.

Cómo activar el modo plan:

```
# En Claude Code: presiona Shift+Tab para entrar a modo plan
# Claude lee archivos y analiza el código SIN modificar nada

# Qué escribirle en modo plan:
Lee src/auth/login.ts y src/middleware/session.ts.
Entiende cómo funciona el flujo de autenticación actual.
Dime qué cambiarías para arreglar el timeout bug.
No toques nada todavía – solo dame el plan.

# Claude responde con el plan. Tú lo apruebas.
# Luego presionas Enter para que ejecute.
```

La diferencia entre modo plan y modo normal: en modo normal Claude asume, actúa y luego te dice qué hizo. En modo plan te dice qué va a hacer antes de tocar nada. Para proyectos con más de 3 archivos involucrados, siempre usa modo plan primero.

TIEMPO: 2 – 5 MIN

PRÁCTICAS 3 – 4

Haz que Claude trabaje solo — y bien

Un buen prompt no es el más largo. Es el que le da a Claude todo lo que necesita para terminar sin preguntarte nada.

3 DALE UNA FORMA DE VERIFICAR SU PROPIO TRABAJO

Si solo le dices qué hacer, Claude termina y dice 'listo'. Sin verificación, no sabe si funcionó. Con un comando de check en el prompt, cierra el loop solo.

Ejemplos concretos de cómo pedirlo:

```
# Para código: incluye el comando de validación
Arregla el bug en src/auth.ts. Cuando termines,
corre npm run test -- auth.test.ts y muéstrame
el output. Si hay errores, corrígelos antes de avisar.

# Para UI: pídele que compare visualmente
Implementa el diseño del header. Después toma
un screenshot con la herramienta browser y
compáralo con la referencia en /docs/design.png.

# Para lógica: dale el resultado esperado
La función debe retornar { status: 'ok', data: [...] }
para este input: [ejemplo]. Verifica que así sea.
```

Sin esto: Claude termina, te dice 'aquí está', y tú tienes que probar manualmente. Con esto: Claude termina, prueba, ve si hay errores y los corrige. Tú solo revisas el resultado final.

TIEMPO: 1 MIN EXTRA EN EL PROMPT

4 ANATOMÍA DE UN PROMPT QUE SÍ FUNCIONA

Un prompt vago produce trabajo vago. Cada dato que omites es una suposición que Claude hace por ti — y casi siempre asume mal. Aquí la estructura que funciona:

```
# MALO — Claude va a adivinar todo:
arregla el bug de login

# BUENO — Claude sabe exactamente qué hacer:
El login falla cuando el token expira después de 30 min.
El error ocurre en src/auth/session.ts línea 47.
El comportamiento esperado: redirigir a /login con
el mensaje 'Tu sesión expiró. Ingresar de nuevo.'
Restricción: no cambies la lógica de refreshToken().
Cuando termines: corre npm run test:auth y muéstrame
el output completo.
```

Fórmula: [qué está pasando] + [dónde exactamente] + [qué debería pasar] + [qué NO tocar] + [cómo verificar]. No necesitas los 5 siempre, pero entre más incluyas, menos va a improvisar.

TIEMPO: 3 – 5 MIN

PRÁCTICAS 5 – 6

Contexto persistente y manejo de alucinaciones

Cómo hacer que Claude sepa quién eres desde el primer mensaje — y qué hacer cuando inventa cosas.

5 CLAUDE.MD = CONTEXTO QUE NO TIENES QUE REPETIR

CLAUDE.md es el archivo que Claude Code lee automáticamente al inicio de cada sesión. Todo lo que pones ahí no tienes que repetirlo nunca más: tu stack, tus comandos, tus reglas de código.

Cómo crearlo (dos formas):

```
# OPCIÓN A – Automático (Claude analiza tu proyecto):  
/init  
# Claude Code escanea tus archivos y genera CLAUDE.md  
# con el stack detectado, scripts de npm y estructura  
  
# OPCIÓN B – Manual. Crea el archivo en la raíz:  
# CLAUDE.md  
Stack: Next.js 14, TypeScript, Tailwind, Supabase  
Comandos útiles: npm run dev | npm run test | npm run check  
Estilo: funciones con arrow functions, no classes  
Siempre corre npm run check antes de avisar que terminaste  
No instales dependencias sin preguntarme primero
```

Qué poner en CLAUDE.md: stack tecnológico, comandos frecuentes, convenciones de código, qué NO hacer (ej: no usar any en TypeScript), y cómo verificar que el trabajo esté bien. Entre más específico, menos supervisión necesitas por sesión.

TIEMPO: 5 – 10 MIN UNA SOLA VEZ

6 CUANDO ALUCINE, PREGÚNTALE QUÉ VIO (NO LO CORRIJAS)

Cuando Claude inventa un nombre de función, un import que no existe o una API que nunca existió, el instinto es corregirlo. Error. Primero descubre de dónde lo sacó.

Qué escribirle exactamente:

```
# Cuando menciona algo que no existe:  
¿De dónde sacaste el nombre 'useAuthSession'?  
¿Qué archivo viste que tenga esa función?  
Muéstrame la línea exacta donde aparece.  
  
# Si no puede mostrártela, sabrá que lo inventó:  
# Claude suele responder 'tienes razón, no existe'  
# y te pide que le digas el nombre correcto.  
  
# NUNCA hagas esto – empeora las cosas:  
# 'No, la función correcta se llama useAuth'  
# (Claude acepta, pero el contexto sigue contaminado)
```

Por qué funciona: las alucinaciones ocurren cuando Claude no tiene la información pero necesita continuar. Preguntarle la fuente lo fuerza a reconocer el vacío de información. Una vez reconocido, puede pedirte el dato correcto en vez de inventar otro.

TIEMPO: 2 MIN

PRÁCTICAS 7 – 8

Tareas largas y revisión de calidad

Cómo estructurar proyectos complejos y usar dos instancias de Claude para que una revise el trabajo de la otra.

7 DIVIDE EN FASES CON CHECKPOINTS EXPLÍCITOS

Una instrucción de 10 pasos produce peor resultado que 10 instrucciones de 1 paso. No porque Claude no entienda — sino porque el contexto se degrada con la longitud.

Cómo estructurarlo en el chat:

```
# Mensaje 1 – solo la fase 1:
Fase 1: Crea la estructura de carpetas del proyecto.
Solo crea los archivos vacíos con los imports base.
No escribas lógica todavía. Avísame cuando termines.

# [Claude termina. Tú revisas y apruebas.]

# Mensaje 2 – solo la fase 2:
Fase 2: Implementa la función fetchUser() en src/api.ts.
Usa el patrón de manejo de errores que ya está en auth.ts.
Corre npm run test:api al terminar. Muéstrame el output.

# Repite por fase. Nunca combines dos fases en un mensaje.
```

Señal de que necesitas dividir más: si tu prompt tiene más de 3 verbos de acción (crea, implementa, conecta, prueba...) es demasiado. Cada mensaje debería tener máximo 2 acciones + 1 verificación.

TIEMPO: VARIABLE — SIEMPRE VALE LA PENA

8 SESIONES PARALELAS: QUE CLAUDE REVISE A CLAUDE

La sesión que escribió el código tiene sesgo hacia ese código — lo defenderá aunque esté mal. Una sesión nueva, sin contexto previo, lo evalúa sin prejuicios. Así funciona:

El flujo exacto:

```
# SESIÓN A – genera el código normalmente
# Cuando termina, copia el código al portapapeles

# SESIÓN B – abre conversación nueva (sin contexto)
Eres un revisor de código senior. Aquí hay código
que escribió otro desarrollador. Necesito que:
1. Identifiques bugs o errores lógicos
2. Señales qué mejorarías en performance
3. Me digas si hay casos edge no manejados
4. Califiques la legibilidad del 1 al 10
No seas condescendiente – sé directo y específico.
[pega el código aquí]

# La Sesión B no sabe quién escribió el código.
# Evalúa sin protegerlo.
```

Bonus: también funciona para prompts. Si escribiste un prompt largo para Claude y quieres saber si está bien estructurado, ábrelo en una sesión nueva y pregúntale: '¿Qué información falta en este prompt para que puedas ejecutarlo sin adivinar nada?'

TIEMPO: 5 – 10 MIN

Nos vemos adentro.

Sigue aprendiendo IA gratis.

Esta guía es el punto de partida. En The Grid encuentras tutoriales paso a paso, workflows reales usados en proyectos con clientes, y una comunidad activa en español que aprende IA contigo. Todo gratis.

COMUNIDAD · THE GRID

Únete gratis a la comunidad

Tutoriales, workflows reales, recursos y una comunidad activa de builders en español aprendiendo IA juntos.

skool.com/the-grid-5633

Diego · [@ortegoat](https://twitter.com/ortegoat)

Co-fundador de The Grid · Educación práctica en IA para negocios en español.

→ Instagram: [@ortegoat](https://twitter.com/ortegoat)

→ Web: ortegoat.com

→ Comunidad: the-grid.mx